Java: Control Statements

Summary — Syeda Ajbina Nusrat, Lecturer, CSE, UITS

Overview

Control statements determine the **order in which code executes**. Without them, Java would run every line from top to bottom with no decisions or repetition.

Category	Purpose	Statements
Sequence	Default top-to-bottom execution	Normal statements
Selection	Choose which block to execute	if, if-else, switch, ?:
Iteration	Repeat a block until condition is met	while, do-while, for
Jump	Transfer control elsewhere immediately	break, continue, return

1. The if Statement

Executes a block **only when a boolean condition is true**. If false, the block is skipped entirely.

Syntax:

```
if (condition) { statement; }
```

Real-world example — checking if a student passed:

```
int score = 75;
if (score >= 50) {
    System.out.println("You passed!");
}
// Output: You passed!
```

■ **Tip:** Always use curly braces { } even for single-line bodies — it prevents bugs when you add more lines later.

2. The if-else Statement

Adds an **alternative branch** — exactly one of the two blocks will always run.

```
int score = 40;
if (score >= 50) {
    System.out.println("Pass");
} else {
    System.out.println("Fail");
}
// Output: Fail
```

3. Nested if…else & the if-else-if Ladder

An if-else placed inside another if-else. Useful for **multiple mutually exclusive conditions**. An **else always pairs with the nearest unmatched if** — use braces to be explicit.

Grading example — if-else-if ladder:

```
int score = 82;
if (score >= 90) {
    System.out.println("Grade: A");
} else if (score >= 80) {
    System.out.println("Grade: B"); // <-- executes this
} else if (score >= 70) {
    System.out.println("Grade: C");
} else {
    System.out.println("Grade: F");
}
```

■ **Pitfall:** The else attaches to the nearest if, not the indented one. Use braces to control which if owns the else.

4. The switch Statement

Evaluates one expression and **jumps to the matching case**. Cleaner than a long if-else ladder when comparing a single value against many options. Expression type must be byte, short, int, char, or String (JDK 7+).

Syntax:

```
switch (expression) {
  case value1: statements; break;
  case value2: statements; break;
  default: statements;
}
```

Day-of-week example:

```
int day = 3;
switch (day) {
  case 1: System.out.println("Monday"); break;
  case 2: System.out.println("Tuesday"); break;
  case 3: System.out.println("Wednesday"); break; // runs this
  default: System.out.println("Other day");
}
// Output: Wednesday
```

Concept	Explanation
break	Exits the switch immediately. Without it, execution 'falls through' to the next case.
default	Optional catch-all executed when no case matches (like the final else).
fall-through	When break is omitted, execution continues into the next case — sometimes useful.

5. The Conditional (Ternary) Operator

A compact one-line alternative to if-else that **returns a value**. Takes three operands: a condition, a true-result, and a false-result.

```
Syntax:   condition ? value_if_true : value_if_false
```

Examples:

```
// Find the larger of two numbers
int a = 10, b = 20;
int max = (a > b) ? a : b;
System.out.println(max);  // Output: 20

// Assign pass/fail label
int score = 65;
String result = (score >= 50) ? "Pass" : "Fail";
System.out.println(result);  // Output: Pass
```

6. Iteration Statements (Loops)

Loops execute a block **repeatedly** until a termination condition is met. Java provides three types — choose based on whether you know the count ahead of time.

Loop	When to use	Executes body at least once?
while	Repeat while a condition is true (count unknown)	No — may execute 0 times
do-while	Same, but always run the body at least once first	Yes — always once
for	Repeat a known number of times (count known in advance)	No — may execute 0 times

6a. The while Loop

```
Syntax:   while (condition) { statement; }
```

Example — print numbers 1 to 5:

```
int count = 1;
while (count <= 5) {
    System.out.println(count); // prints 1, 2, 3, 4, 5
    count++;
}
```

■ **Pitfall:** If you forget count++, the condition is always true → **infinite loop**!

6b. The do-while Loop

```
Syntax:   do { statement; } while (condition);
```

Example — menu that shows at least once:

```
int choice;
do {
    System.out.println("Enter 1-Exit, 2-Continue:");
    choice = scanner.nextInt();
} while (choice != 1);  // keeps looping until user enters 1
```

■ **Key difference from while:** do-while checks the condition *after* executing, so the body always runs at least once — ideal for input validation menus.

6c. The for Loop

```
Syntax:   for (initialization; condition; increment) { statement; }
```

Example — print multiplication table of 3:

```
for (int i = 1; i <= 10; i++) {
    System.out.println("3 x " + i + " = " + (3*i));
}
// Output: 3 x 1 = 3, 3 x 2 = 6, ..., 3 x 10 = 30
```

Part	Role	If omitted...
initialization	Runs once before the loop starts	No initialization performed
condition	Checked before each iteration	Treated as true → infinite loop
increment	Runs at the end of each iteration	No increment performed

Note: Both semicolons in the for header are always required.

6d. Nested Loops

A loop inside another loop. Each full pass of the outer loop triggers a **complete run** of the inner loop. Common for grids, tables, and patterns.

Example — print a 3×3 multiplication grid:

```
for (int i = 1; i <= 3; i++) {
  for (int j = 1; j <= 3; j++) {
    System.out.print(i*j + "\t");
  }
  System.out.println();
}
// Output:
// 1 2 3
// 2 4 6
// 3 6 9
```

7. Jump Statements

Statement	Effect
break	Exits the current loop or switch immediately
continue	Skips the rest of the current iteration; moves to the next one
return	Exits the current method immediately (optionally returns a value)

break example — stop at first even number:

```
for (int i = 1; i <= 10; i++) {
  if (i % 2 == 0) {
    System.out.println("First even: " + i); // prints 2
    break;  // exits loop immediately
  }
}
```

continue example — skip odd numbers (print only evens):

```
for (int i = 1; i <= 6; i++) {
  if (i % 2 != 0) continue;  // skip odd numbers
  System.out.println(i);
}
// Output: 2, 4, 6
```

return example — exit a method early:

```
public static void checkAge(int age) {
  if (age < 18) {
    System.out.println("Access denied.");
    return;  // method exits here
  }
  System.out.println("Access granted."); // only reached if age >= 18
}
```

■ **Labeled break/continue:** In nested loops, you can label the outer loop and use **break label;** or **continue label;** to jump out of/continue the outer loop directly, not just the inner one.

Quick Reference — Choosing the Right Tool

Situation	Best choice
One condition, do something or skip	if
One condition, do one of two things	if-else
3+ conditions on the same variable (int/char)	switch
Multiple ranges or complex conditions	if-else-if ladder
One-line value decision	?: ternary
Loop, condition known, count unknown	while
Loop that must run at least once	do-while
Loop with a known count	for
Exit loop/switch early	break
Skip current iteration	continue